`random` and `math`

Status: not covered by Software Lab-1 practicals **Where it is used:** `scripts/build_synthetic_registries.py` (`random.Random`, `random.choice`, `randint`, `gauss`, `uniform`, `randrange`; `math.sqrt`) **Closest lab concept:** Practical 2, control structures - a `for` loop over a fixed `range()` gives every record the same value unless something varies it; `random` is that variation, and `math` is the arithmetic on the result

---

1. What they are

Two stdlib modules, both shipped with Python.

`random` produces numbers from a sequence you cannot predict without knowing the seed. `random.random()` gives a float between 0 and 1. `random.randint(2, 7)` gives a whole number from 2 to 7 inclusive. `random.choice([...])` picks one item from a list, which is how a patient gets a blood group or a name.

`math` is the arithmetic library: square roots, logarithms, trigonometry. The project needs exactly one function from it, `math.sqrt`, because height in centimetres has to be turned into BMI and BMI is weight divided by height in metres, squared.

Both are stdlib, like `os`, `csv` and `json`. `random` is not randomness in the cryptographic sense and must never be used for a password or a token. It is a generator, and it is exactly repeatable, which is the property this project depends on.

---

2. Why the project needed them

The seed registries hold 520 hospitals, 540 doctors, 600 patients, 2,076 visits and 520 pharmacies. Writing those by hand is not possible, and writing them with a loop that assigns the same value to every row is worse than useless, because the result is a CSV of 600 identical people.

`random` is what makes each row differ while the whole file stays reproducible.

---

3. The part that actually matters: seeding

`random` without a seed gives a different file every run. That is fatal for a checked-in dataset, because:

- `git diff` would show every row changing on a no-op regeneration;
- a bug report could not be reproduced, because the data that caused it no longer exists;
- the integrity checker and the test suite would be testing different data each time.

The fix is a seed.

```
rng = random.Random(SEED)
```

`random.Random(seed)` builds a *private* generator. Every value it produces is determined by the seed alone, so the same seed always yields the same sequence. The generator uses one master seed and derives a separate stream per record:

```
for index in range(first, N_PATIENTS + 1):
    rng = random.Random(SEED * 3000 + index)
```

The reason for a per-record stream rather than one shared generator is subtle and worth stating. If every record drew from a single `Random(SEED)` in sequence, then adding one row near the start would shift every later draw, and the whole file would change. Giving each record its own generator keyed on its

index means row 400 is identical whether the file has 400 rows or 600. Regeneration stays local and reviewable.

`SEED = 20261004` is the date this was written, chosen so the value is obviously deliberate rather than a magic number.

---

4. Distribution control

Plain `random` is uniform: every option is equally likely. Real data is not uniform, and uniform data is visibly wrong. An age profile that produces equal numbers of infants and 90-year-olds is not a plausible population.

The generator therefore uses **weighted choice**, built from a list of `(value, weight)` pairs:

```
GENDERS = [("female", 49), ("male", 51)]

def weighted_choice(rng, pairs):
    total = sum(weight for _value, weight in pairs)
    roll = rng.uniform(0, total)

    for value, weight in pairs:
        if roll < weight:
            return value
        roll -= weight
```

`rng.uniform(0, total)` picks a point on the 0-to-total line, then each entry claims a slice of it proportional to its weight. Female patients occupy 49 of the 100 units, so they come out slightly under half the time. Change the weights and the population profile changes; that is the only reason the weights are written down as data.

The same helper gives Nagpur the largest share of the empanelled network and a rare diagnosis the smallest share of visits.

---

5. Why `math.sqrt` and not `** 0.5`

BMI is weight in kilograms over height in metres, squared:

```
bmi = weight / ((height / 100) ** 2)
```

`height` is stored in centimetres in the CSV because that is how a nurse writes it down, so it is divided by 100 first. `math.sqrt(x)` and `x ** 0.5` are the same operation; `math.sqrt` is used where the intent is "this is a square root" rather than "this is a power", because it says so out loud.

BMI is then banded against the WHO adult cut-offs, under 18.5 underweight, 18.5 to 24.9 normal, 25 to 29.9 overweight, 30 and above obese.

---

6. Traps worth knowing

`random.choice` on a generator expression. Passing `rng.choice(x for x in y)` exhausts nothing, but passing `rng.choice(x for x in y)` twice over the *same* generator object silently returns `None` the second time. Materialise it as a list first.

Weighting is not sampling. `weighted_choice` above walks the list once per call. For 600 patients that is nothing. For 100,000 rows it would not be, and the cumulative-weight form using `itertools.accumulate` and `bisect` would be the right answer.

Clamping silently changes the distribution. Vitals are clamped into a survivable range with `max(88, min(196, value))`. That guarantees the integrity checker passes, and it also quietly hides the fact that the underlying distribution had a tail outside human range. The clamp is there on purpose and the base distribution is tight enough that the clamp rarely bites, but a clamp that is doing real work is a signal that

the distribution is wrong.

`random` is not for secrets. `random.choice` over an alphabet is predictable from a handful of outputs. Anything security-related needs `secrets`, which is a different module for exactly this reason.